# Program Analysis 2025-26

## Lecture #20

## Control Flow Analysis


Università di Pisa

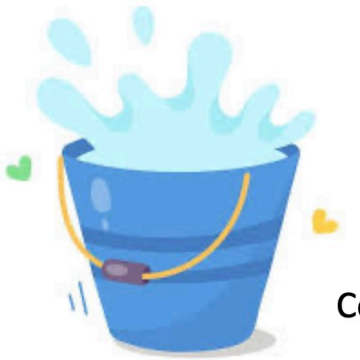
# Constraint Based 0-CFA Analysis

# Static Analysis: a constraint-based approach

**The basic idea is to separate the analysis specification from algorithmic aspects and implementation details**

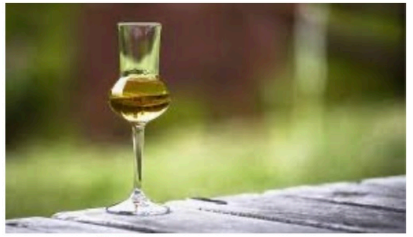
Program to analyze

The analysis



Constraints

Constraint Solver



SOLUTION

# Constraint Based 0-CFA Analysis

- We are now ready to compute the least solution $(\hat{C}, \hat{\rho})$ such that
- We have learnt that constraint generation is a syntax driven approach
- We turn semantic rules into constraints
- To this aim, we first construct a finite set $C_*[\![e_*]\!]$ of *constraints* and *conditional constraints* $ls$ in the form:

$$lhs \subseteq rhs$$

$$\{t\} \subseteq rhs' \Rightarrow lhs \subseteq rhs$$

if t may flow to rhs', then lhs must flow to rhs

where:

- $lhs ::= \hat{C}(\ell) \mid \hat{\rho}(x), \{t\}$ where $t$ is a function abstraction
- $rhs := \hat{C}(\ell) \mid \hat{\rho}(x)$

# From rules to constraints

- Expand each clause defining $(\hat{C}, \hat{\rho}) \vDash_s e_*$ into a finite set of constraints and
- Collect all constraints into $C_*[\![e_*]\!]$

logical formula

set of syntactic entities

**Idea**:
Occurrences of $\hat{C}$ are changed into $C$ and
occurrences of $\hat{\rho}$ are changed into $r$

$C$ and $r$ are the constraint variables representing the unknown solution

The function $C_*$ given an expression $e_*$ returns a set of constraints $C_*[\![e_*]\!]$
We generate constraints from the syntax-directed rules

# Clauses for constants

Constants generate no constraints

$$[con]\ (\hat{C}, \hat{\rho}) \vDash_s c^{\ell} \text{ iff } \varnothing \subseteq \hat{C}(\ell)$$

$$C_*[\![c^{\ell}]\!] = \varnothing$$

constants generate no constraints:

**Example**

$e \triangleq 7^1$

$C_*[\![7^1]\!] = \varnothing$

# Clauses for variables

Variables generate a constraint reflecting a flow from $r(x)$ to $C(\ell)$

$$[var]\ (\hat{C}, \hat{\rho}) \vDash_s x^{\ell} \text{ iff } \hat{\rho}(x) \subseteq \hat{C}(\ell)$$

$$C_*[\![x^{\ell}]\!] = \{r(x) \subseteq C(\ell)\}$$

**Example**

$e \triangleq \mathrm{x}^1$

$C_*[\![\mathrm{x}^1]\!] = \{r(\mathrm{x}) \subseteq C(1)\}$

# Clauses for functions

Function abstractions generate:

- a flow constraint, and, recursively,
- constraints from the body

$$[fn]\ (\hat{C},\hat{\rho}) \vDash_s (\text{fn } x \Rightarrow e_0)^{\ell} \text{ iff } \{\text{fn } x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \wedge (\hat{C},\hat{\rho}) \vDash_s e_0$$

$$C_*[\![(\text{fn } x \Rightarrow e_0)^{\ell}]\!] = \{\{\text{fn } x \Rightarrow e_0\} \subseteq C(\ell)\} \cup C_*[\![e_0^{\ell}]\!]$$

**Example**

$e \triangleq (\text{fn x => x}^1)^2$

function constraint

Body constraint

$$C_*[\![(\text{fn x => x}^1)^2]\!] = \{\{\text{fn x => x}^1\} \subseteq C(2)\} \cup \{r(\text{x}) \subseteq C(1)\}$$

# Clauses for recursive functions

Function abstractions generate:

- a flow constraint, and, recursively,
- constraints from the body

$$[fun]\ (\hat{C},\hat{\rho}) \vDash_s (\text{fun f } x \Rightarrow e_0)^{\ell} \text{ iff } \{\text{fun f } x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \wedge (\hat{C},\hat{\rho}) \vDash_s e_0 \wedge \{\text{fun f } x \Rightarrow e_0\} \subseteq \hat{\rho}(f)$$

$$C_*[\![(\text{fun f } x \Rightarrow e_0)^{\ell}]\!] = \{\{\text{fun f } x \Rightarrow e_0)^{\ell}\} \subseteq C(\ell)\} \cup C_*[\![e_0^{\ell}]\!] \cup \{\{\text{fun f } x \Rightarrow e_0)^{\ell}\} \subseteq r(f)\}$$

recursive binding: function flows to f

# Clause for function application

Application generates recursively constraints from its terms plus the body conditional constraints

$$[app]\ (\hat{C},\hat{\rho}) \vDash_s (t_1^{\ell_1} t_2^{\ell_2})^{\ell} \text{ iff } (\hat{C},\hat{\rho}) \vDash_s t_1^{\ell_1} \wedge (\hat{C},\hat{\rho}) \vDash_s t_2^{\ell_2}$$
$$(\forall(\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$$
$$\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge\ \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge \ldots$$

$$C_*[\![(t_1^{\ell_1} t_2^{\ell_2})^{\ell}]\!] = C_*[\![t_1^{\ell_1}]\!] \cup C_*[\![t_2^{\ell_2}]\!] \cup$$
$$\{\{t\} \subseteq C(\ell_1) \Rightarrow C(\ell_2) \subseteq r(x) \,|\, t = (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \mathbf{Term}_*\} \cup$$
$$\{\{t\} \subseteq C(\ell_1) \Rightarrow C(\ell_0) \subseteq C(\ell) \,|\, t = (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \mathbf{Term}_*\} \cup \ldots$$

Conditional constraints

For each possible function $t$ at $\ell_1$ in $\mathbf{Term}_*$: if $t$ flows to $\ell_1$, then:

- argument flows to parameter
- result flows to $\ell$

The set of function abstractions in the terms to analyze

# Clause for function application

$$[app]\ (\hat{C},\hat{\rho}) \models_s (t_1^{\ell_1} t_2^{\ell_2})^{\ell} \text{ iff } (\hat{C},\hat{\rho}) \models_s t_1^{\ell_1} \wedge (\hat{C},\hat{\rho}) \models_s t_2^{\ell_2}$$
$$(\forall(\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$$
$$\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$$
$$(\forall(\text{fun f } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$$
$$\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell))$$

$$C_*[\![(t_1^{\ell_1} t_2^{\ell_2})^{\ell}]\!] = C_*[\![t_1^{\ell_1}]\!] \cup C_*[\![t_2^{\ell_2}]\!] \cup \{\{t\} \subseteq C(\ell_1) \Rightarrow C(\ell_2) \subseteq r(x) \mid t = (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \mathbf{Term}_*\}$$
$$\cup \{\{t\} \subseteq C(\ell_1) \Rightarrow C(\ell_0) \subseteq C(\ell) \mid t = (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \mathbf{Term}_*\}$$
$$\cup \{\{t\} \subseteq C(\ell_1) \Rightarrow C(\ell_2) \subseteq r(x) \mid t = (\text{fun f } x \Rightarrow t_0^{\ell_0}) \in \mathbf{Term}_*\}$$
$$\cup \{\{t\} \subseteq C(\ell_1) \Rightarrow C(\ell_0) \subseteq C(\ell) \mid t = (\text{fun f } x \Rightarrow t_0^{\ell_0}) \in \mathbf{Term}_*\}$$

See **Example 1** next

# Constraint based CFA

$$
\begin{array}{ll}
[con] & \mathcal{C}_\star[\![c^\ell]\!] = \emptyset \\
[var] & \mathcal{C}_\star[\![x^\ell]\!] = \{\mathsf{r}(x) \subseteq \mathsf{C}(\ell)\} \\
[fn] & \mathcal{C}_\star[\![(\texttt{fn}\ x \texttt{ => } e_0)^\ell]\!] = \{\{\texttt{fn}\ x \texttt{ => } e_0\} \subseteq \mathsf{C}(\ell)\} \\
 & \qquad \cup\, \mathcal{C}_\star[\![e_0]\!] \\
[fun] & \mathcal{C}_\star[\![(\texttt{fun}\ f\ x \texttt{ => } e_0)^\ell]\!] = \{\{\texttt{fun}\ f\ x \texttt{ => } e_0\} \subseteq \mathsf{C}(\ell)\} \\
 & \qquad \cup\, \mathcal{C}_\star[\![e_0]\!] \cup \{\{\texttt{fun}\ f\ x \texttt{ => } e_0\} \subseteq \mathsf{r}(f)\} \\
[app] & \mathcal{C}_\star[\![(t_1^{\ell_1}\ t_2^{\ell_2})^\ell]\!] = \mathcal{C}_\star[\![t_1^{\ell_1}]\!] \cup \mathcal{C}_\star[\![t_2^{\ell_2}]\!] \\
 & \qquad \cup \{\{t\} \subseteq \mathsf{C}(\ell_1) \Rightarrow \mathsf{C}(\ell_2) \subseteq \mathsf{r}(x) \\
 & \qquad\quad \mid t = (\texttt{fn}\ x \texttt{ => } t_0^{\ell_0}) \in \mathbf{Term}_\star\} \\
 & \qquad \cup \{\{t\} \subseteq \mathsf{C}(\ell_1) \Rightarrow \mathsf{C}(\ell_0) \subseteq \mathsf{C}(\ell) \\
 & \qquad\quad \mid t = (\texttt{fn}\ x \texttt{ => } t_0^{\ell_0}) \in \mathbf{Term}_\star\} \\
 & \qquad \cup \{\{t\} \subseteq \mathsf{C}(\ell_1) \Rightarrow \mathsf{C}(\ell_2) \subseteq \mathsf{r}(x) \\
 & \qquad\quad \mid t = (\texttt{fun}\ f\ x \texttt{ => } t_0^{\ell_0}) \in \mathbf{Term}_\star\} \\
 & \qquad \cup \{\{t\} \subseteq \mathsf{C}(\ell_1) \Rightarrow \mathsf{C}(\ell_0) \subseteq \mathsf{C}(\ell) \\
 & \qquad\quad \mid t = (\texttt{fun}\ f\ x \texttt{ => } t_0^{\ell_0}) \in \mathbf{Term}_\star\} \\
[if] & \mathcal{C}_\star[\![(\texttt{if}\ t_0^{\ell_0}\ \texttt{then}\ t_1^{\ell_1}\ \texttt{else}\ t_2^{\ell_2})^\ell]\!] = \mathcal{C}_\star[\![t_0^{\ell_0}]\!] \cup \mathcal{C}_\star[\![t_1^{\ell_1}]\!] \cup \mathcal{C}_\star[\![t_2^{\ell_2}]\!] \\
 & \qquad \cup \{\mathsf{C}(\ell_1) \subseteq \mathsf{C}(\ell)\} \\
 & \qquad \cup \{\mathsf{C}(\ell_2) \subseteq \mathsf{C}(\ell)\} \\
[let] & \mathcal{C}_\star[\![(\texttt{let}\ x = t_1^{\ell_1}\ \texttt{in}\ t_2^{\ell_2})^\ell]\!] = \mathcal{C}_\star[\![t_1^{\ell_1}]\!] \cup \mathcal{C}_\star[\![t_2^{\ell_2}]\!] \\
 & \qquad \cup \{\mathsf{C}(\ell_1) \subseteq \mathsf{r}(x)\} \cup \{\mathsf{C}(\ell_2) \subseteq \mathsf{C}(\ell)\} \\
[op] & \mathcal{C}_\star[\![(t_1^{\ell_1}\ op\ t_2^{\ell_2})^\ell]\!] = \mathcal{C}_\star[\![t_1^{\ell_1}]\!] \cup \mathcal{C}_\star[\![t_2^{\ell_2}]\!]
\end{array}
$$

# Back to Example 1

We generate the following set of constraints for our expression $((\texttt{fn x => x}^1)^2\ (\texttt{fn y => y}^3)^4)^5$

$C_*[\![e_*]\!] = C_*[\![(\texttt{fn x => x}^1)^2]\!] \cup C_*[\![(\texttt{fn y => y}^3)]\!] \cup$ [app]

$\{\texttt{fn y => y}^3\} \in \mathbf{Terms}_*$

$\{\texttt{fn x => x}^1\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$

$\{\texttt{fn x => x}^1\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$

$\{\texttt{fn y => y}^3\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$

$\{\texttt{fn y => y}^3\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$

| | $(\widehat{C}_e, \widehat{\rho}_e)$ |
|---|---|
| 1 | $\{\texttt{fn y => y}^3\}$ |
| 2 | $\{\texttt{fn x => x}^1\}$ |
| 3 | $\emptyset$ |
| 4 | $\{\texttt{fn y => y}^3\}$ |
| 5 | $\{\texttt{fn y => y}^3\}$ |
| x | $\{\texttt{fn y => y}^3\}$ |
| y | $\emptyset$ |

[fn]

$C_*[\![(\texttt{fn x => x}^1)^2]\!] = \{\texttt{fn x => x}^1\} \subseteq C(2) \cup C_*[\![\texttt{x}^1]\!]$

$C_*[\![(\texttt{fn y => y}^3)^4]\!] = \{\texttt{fn y => y}^3\} \subseteq C(4) \cup C_*[\![\texttt{y}^3]\!]$

[var]

$C_*[\![\texttt{x}^1]\!] = r(x) \subseteq C(1)$

$C_*[\![\texttt{y}^3]\!] = r(y) \subseteq C(3)$

We use that $\texttt{fn x => x}^1$ and $\texttt{fn y => y}^3$ are the only abstractions in $\mathbf{Term}_*$

# Back to Example 1

We generate the following set of constraints for our expression $((\texttt{fn x => x}^1)^2\ (\texttt{fn y => y}^3)^4)^5$

| | $(\widehat{C}_e, \widehat{\rho}_e)$ |
|---|---|
| 1 | $\{\texttt{fn y => y}^3\}$ |
| 2 | $\{\texttt{fn x => x}^1\}$ |
| 3 | $\emptyset$ |
| 4 | $\{\texttt{fn y => y}^3\}$ |
| 5 | $\{\texttt{fn y => y}^3\}$ |
| x | $\{\texttt{fn y => y}^3\}$ |
| y | $\emptyset$ |

$\{\texttt{fn x => x}^1\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$
$\{\texttt{fn x => x}^1\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$
$\{\texttt{fn y => y}^3\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$
$\{\texttt{fn y => y}^3\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$
$\{\texttt{fn x => x}^1\} \subseteq C(2)$
$\{\text{fn y => y}^3\} \subseteq C(4)$
$r(x) \subseteq C(1)$
$r(y) \subseteq C(3)$

# Complexity

The function $C^*$ uses the set of terms $\mathbf{Term}_*$. Let $n$ be the size of the expression $e_*$

the number of sub-terms

**Naive estimate**:

- $O(n^2)$ constraints
- $O(n^4)$ conditional constraints

**Refined estimate**:

Key observation: Every construct, except applications, gives us only one constraint.

Applications may gives us $O(n)$ constraints:

- $O(n)$ constraints
- $O(n^2)$ conditional constraints

Each of the $O(n)$ sub-terms only generate $O(1)$ constraints and $O(n)$ conditional constraints

The analysis remains polynomial despite higher-order functions

# Back to Example 1

$$((\texttt{fn x => x}^{1})^{2}\ (\texttt{fn y => y}^{3})^{4})^{5}$$

$n = 5$

**Refined estimate**:

- $O(n) = 5$ constraints
- $O(n^2) = 25$ conditional constraints

worst-case bounds, but not reached here

Here, we just have **two functions** and **one application**. For each $t \in C(2)$ we generate two conditional constraints. For each subterm we only generate one constraint

- 4 constraints
- 4 conditional constraints

We use that fn x => $\texttt{x}^{1}$ and fn y => $\texttt{y}^{3}$ are the only abstractions in $\mathbf{Term}_*$

| | |
|---|---|
| $\{\text{fn x => x}^{1}\} \subseteq C(2)$ | $\{\texttt{fn x => x}^{1}\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$ |
| $\{\text{fn y => y}^{3}\} \subseteq C(4)$ | $\{\texttt{fn x => x}^{1}\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$ |
| $r(x) \subseteq C(1)$ | $\{\texttt{fn y => y}^{3}\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$ |
| $r(y) \subseteq C(3)$ | $\{\texttt{fn y => y}^{3}\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$ |

# From rules to constraints

To give meaning to the syntax, we can translate syntactic entities to sets of terms:

$(\hat{C}, \hat{\rho})[\![C(\ell)]\!] = \hat{C}(\ell)$

$(\hat{C}, \hat{\rho})[\![r(x)]\!] = \hat{\rho}(x)$

$(\hat{C}, \hat{\rho})[\![\{t\}]\!] = \{t\}$

$$(\hat{C}, \hat{\rho})[\![\{t\} \subseteq rhs' \Rightarrow lhs]\!] = \begin{cases} (\hat{C}, \hat{\rho})[\![lhs]\!] & \text{if } \{t\} \in (\hat{C}, \hat{\rho})[\![rhs']\!] \\ \varnothing & \text{otherwise} \end{cases}$$

Satisfaction relation for constraints:

$(\hat{C}, \hat{\rho})[\![C(\ell)]\!] \vDash_c (ls \subseteq rhs)$ iff $(\hat{C}, \hat{\rho})[\![ls]\!] \subseteq (\hat{C}, \hat{\rho})[\![rhs]\!]$

$(\hat{C}, \hat{\rho})[\![C(\ell)]\!] \vDash_c C$ iff $\forall (ls \subseteq rhs) \in C : (\hat{C}, \hat{\rho}) \vDash_c (ls \subseteq rhs)$

set of constraints

# Preservation of solutions

- All solutions to the set $C_*[\![e_*]\!]$ of constraints also satisfy the syntax directed specification of the CFA and vice versa (when restricted to program terms)
- Thus computing the least solution of the constraints $C_*[\![e_*]\!]$ gives exactly the least CFA solution $(\hat{C}, \hat{\rho})$ to $(\hat{C}, \hat{\rho}) \models_s e_*$

**Proposition**
If $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}^\top, \hat{\rho}^\top)$
then $(\hat{C}, \hat{\rho}) \models_s e_*$ iff $(\hat{C}, \hat{\rho}) \models_c C_*[\![e_*]\!]$

# Preservation of solutions

- All solutions to the set $C_*[\![e_*]\!]$ of constraints also satisfy the syntax directed specification of the CFA and vice versa (when restricted to program terms)
- Thus computing the least solution of the constraints $C_*[\![e_*]\!]$ gives exactly the least CFA solution $(\hat{C},\hat{\rho})$ to $(\hat{C},\hat{\rho}) \models_s e_*$

**Proposition**
If $(\hat{C},\hat{\rho}) \subseteq (\hat{C}^\top,\hat{\rho}^\top)$
then $(\hat{C},\hat{\rho}) \models_s e_*$ iff $(\hat{C},\hat{\rho}) \models_c C_*[\![e_*]\!]$

**Proof sketch**
A simple structural induction on $e$ shows that $(\hat{C},\hat{\rho}) \models_s e$ iff $(\hat{C},\hat{\rho}) \models_c C_*[\![e]\!]$ for all sub-terms $e$ of $e_*$.
In the case of function application $(t_1^{\ell_1} t_2^{\ell_2})^{\ell}$ the assumption $(\hat{C},\hat{\rho}) \subseteq (\hat{C}^\top,\hat{\rho}^\top)$ is used to ensure that $\hat{C}(\ell_1) \subseteq \mathbf{Terms}_*$

# Solving the Constraints

Finding the least solution of the constraints amounts to computing the least fixed point of a function

1. Define a suitable monotone function $F_*$ over analyses that propagates constraints over $C$ and $r$

$$F_* : \widehat{\mathbf{Cache}}_* \times \widehat{\mathbf{Env}}_* \to \widehat{\mathbf{Cache}}_* \times \widehat{\mathbf{Env}}_*$$

   show that it has a least fixed point $lfp(F_*)$ that is indeed the least solution
2. Compute the $lfp(F_*)$

Straightforward techniques allow us to compute the least fixed point in $O(n^5)$ time when $n$ is the size of the expression $e_*$

But we can do better, using **graph formulation of the constraints**: we obtain $O(n^3)$

# Solving the Constraints

We can visualize a constraint system as a constraint graph:

**Nodes**

- a node $C(\ell)$ for each label $\ell \in \mathbf{Lab}_*$
- a node $r(x)$ for each variable $x \in \mathbf{Var}_*$

**Edges**

- Each constraint $p_1 \subseteq p_2$ adds an initial edge $(p_1, p_2)$ to the graph
- Each constraint $\{t\} \subseteq p \Rightarrow p_1 \subseteq p_2$ adds
  - a candidate edge $(p_1, p_2)$, with "trigger" $\{t\} \subseteq p$, and
  - an edge $(p, p_2)$

**Example 1**

$$\begin{array}{c} \{id_x\} \subseteq C(2) \\ \{id_y\} \subseteq C(4) \\ r(x) \subseteq C(1) \\ r(y) \subseteq C(3) \end{array}$$

$$\begin{array}{l} \{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x) \\ \{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5) \\ \{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y) \\ \{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5) \end{array}$$

# Solving the Constraints

We can visualize a constraint system as a constraint graph:

**Nodes**

- a node $C(\ell)$ for each label $\ell \in \mathbf{Lab}_*$
- a node $r(x)$ for each variable $x \in \mathbf{Var}_*$

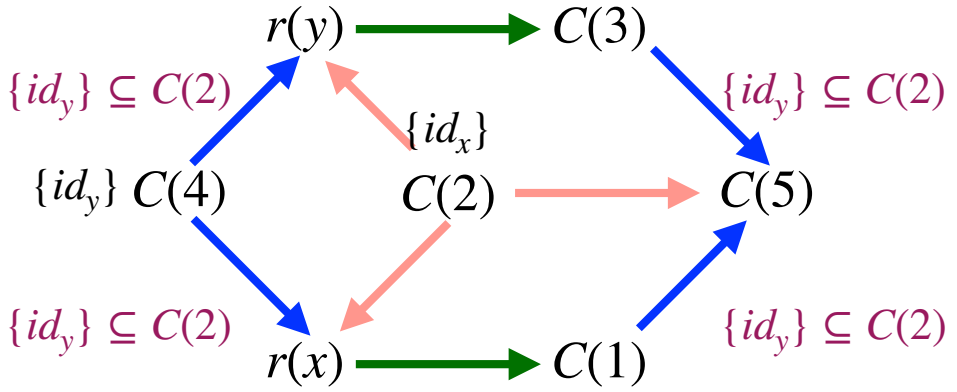
**Edges**

- Each constraint $p_1 \subseteq p_2$ adds an initial edge $(p_1, p_2)$ to the graph
- Each constraint $\{t\} \subseteq p \Rightarrow p_1 \subseteq p_2$ adds
  - a candidate edge $(p_1, p_2)$, with "trigger" $\{t\} \subseteq p$, and
  - an edge $(p, p_2)$

**Example 1**

$\{id_x\} \subseteq C(2)$
$\{id_y\} \subseteq C(4)$
$r(x) \subseteq C(1)$
$r(y) \subseteq C(3)$

$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$
$\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$
$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$
$\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$



# Solving the Constraints: idea

Associated with each node $p$ we have a data field $D[p]$ representing the current knowledge. Initially $D[p]$ is given by

$$D[p] = \{t \mid (\{t\} \subseteq p) \in C_*[\![e_*]\!]\}$$

All edges are traversed in order to propagate information from one data field to another

# Value sets

Each node p of the constraint graph is associated with a set (Data array) $D[p]$ of values

**Goal**: find the value set of every node

syntactical representations of functions

Initially, we have

$$D[p] = \{t \mid (\{t\} \subseteq p) \in C_*[\![e_*]\!]\}$$

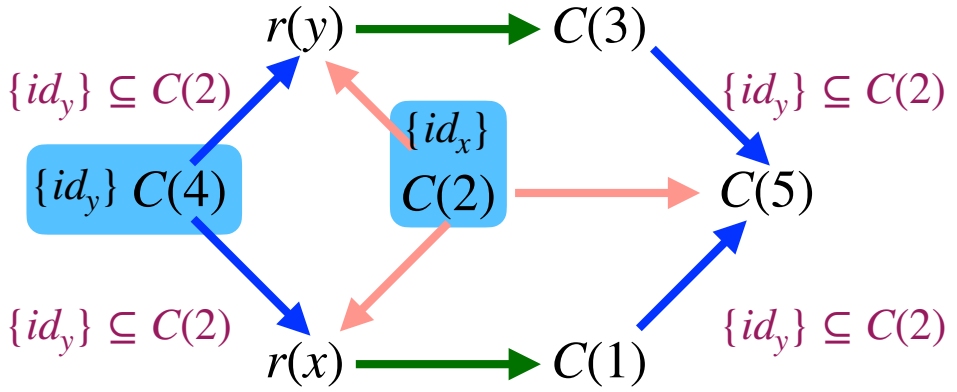
**Example 1**

$$D[C(2)] = \{id_x\}$$
$$D[C(4)] = \{id_y\}$$
$$D[p] = \varnothing \text{ for other nodes } p$$

**Example 1**

$\{id_x\} \subseteq C(2)$
$\{id_y\} \subseteq C(4)$
$r(x) \subseteq C(1)$
$r(y) \subseteq C(3)$

$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$
$\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$
$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$
$\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$



Constraint solving = graph reachability + propagation

# Propagation

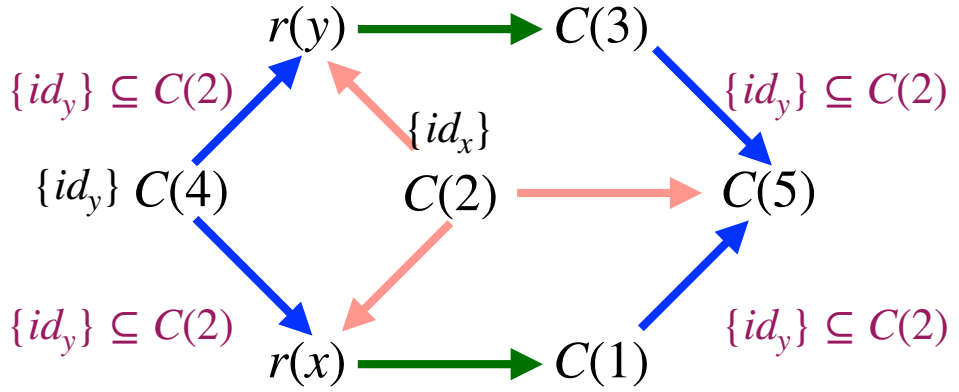
The idea of the algorithm is to iteratively propagate the information from one data field to another according to the constraints

- If we have an edge $(p_1, p_2)$ in our graph, then the value set of $p_1$ must flow into the value set of $p_2$
- If a trigger is true then also in the corresponding edge the value set flows

**Example 1**

$$\{id_x\} \subseteq C(2)$$
$$\{id_y\} \subseteq C(4)$$
$$r(x) \subseteq C(1)$$
$$r(y) \subseteq C(3)$$

$$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$$
$$\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$$
$$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$$
$$\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$$



# The Constraint Solver

**Input**: a set of constraint $C_*[\![e_*]\!]$

**Output**: the least solution $(\hat{C}, \hat{\rho})$ to the constraints (an assignment to variables $C(\ell)$ and $r(x)$

**Data structures**:

- a worklist **W** containing the nodes whose data array should be propagated (a list of nodes whose outgoing edges should be traversed)
- a data array **D** storing for each node the current abstract value in $\mathbf{Val}_*$
- an edge array **E** storing for each node a list of the constraints that the node may affect (i.e., a list of constraints from which a list of the successor nodes can be computed)

The nodes consist $C(\ell)$ for all labels $\ell$ and $r(x)$ for all labels $x$

The solver computes a least fixed point by propagating values until saturation

# The algorithm

Constraint solving = graph reachability + propagation

**Idea**

Each node stores a set of abstract values, and edges describe how these values must be propagated

The algorithm propagates abstract values along edges until no new information appears

- $D$ = current knowledge
- $E$ = how knowledge propagates
- $W$ = what still needs to be processed

# The algorithm: step 1

Initialize all the data structures $D$, $E$ and $W$ to [ ]

# The algorithm: step 2

Build the graph:

**Nodes**

- a node $C(\ell)$ for each label $\ell \in \mathbf{Lab}_*$
- a node $r(x)$ for each variable $x \in \mathbf{Var}_*$

**Edges**

$E[p_1] := \text{cons}(p_1 \subseteq p_2, E[p_1])$

$E[p_1] := \text{cons}(\{t\} \subseteq p \Rightarrow p_1 \subseteq p_2, E[p_1])$

$E[p] := \text{cons}(\{t\} \subseteq p \Rightarrow p_1 \subseteq p_2, E[p])$

- Each constraint $p_1 \subseteq p_2$ adds an initial edge $(p_1, p_2)$ to the graph
- Each constraint $\{t\} \subseteq p \Rightarrow p_1 \subseteq p_2$ adds a candidate edge $(p_1, p_2)$, with "trigger" $\{t\} \subseteq p$, and an edge $(p, p_2)$

**Initial assignments to** $D$: if $(\{t\} \subseteq p) \in C_*[\![e_*]\!]$ and $\{t\} \notin D[p]$: add(p, {t})

- incorporates $\{t\}$ into $D[p]$ and
- adds p to the worklist $W$

$D[p] = \{t \mid (\{t\} \subseteq p) \in C_*[\![e_*]\!]\}$

# The algorithm: step 3

Continue propagating the information from one data field to another, along edges, as long as $W$ is non-empty

$E[q]$: list of the constraints that $q$ may affect

We extract the head element $q$ of $W$ and for all constraints $cc$ in $E[q]$

- $cc = p_1 \subseteq p_2$: If we have an edge $(p_1, p_2)$ in our graph , then the value set of $p_1$ must flow into the value set of $p_2$ (add$(p_2, D[p_1])$)
- $cc = \{t\} \subseteq p \Rightarrow p_1 \subseteq p_2$: If the trigger is true ($t \in D[p]$), then also in the corresponding edge the value set flows (add$(p_2, D[p_1])$)

# The algorithm: step 4

Record the solution in a more familiar form:

- $\forall \ell \in \mathbf{Lab}_* : \hat{C}(\ell) := D[C(\ell)]$
- $\forall x \in \mathbf{Var}_* : \hat{\rho}(x) := D[r(x)]$

# The algorithm pseudocode (1)

```
(* Step 1: Initialization *)
W := [];
for q in Vars do
D[q] := [];
E[q] := [];
done
```

# The algorithm pseudocode (2)

```
(* Step 2: Build the graph *)
for cc in C*[[e*]] do
match cc with
| {t} ⊆ p -> add(p,{t})
|  p1 ⊆ p2 -> E[p1] := cc :: E[p1]
| {t} ⊆ p => p1 ⊆ p2 -> E[p1] := cc :: E[p1];
                        E[p]  := cc :: E[p];
done
```

```
let add(q,d) =
    if ¬ (d ⊆ D[q])
    then D[q] := D[q] U d;
    W := push W q;
```

# Back to ex. 1: step 2

Initially, we have

$D[p] = \{t \mid (\{t\} \subseteq p) \in C_*[\![e_*]\!]\}$

Current abstract values

$\{id_x\} \subseteq C(2)$
$\{id_y\} \subseteq C(4)$
$r(x) \subseteq C(1)$
$r(y) \subseteq C(3)$

$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$
$\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$
$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$
$\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$

```
(* Step 2: Build the graph *)
for cc in C*[[e*]] do
match cc with
| {t} ⊆ p -> add(p,{t})
|  p1 ⊆ p2 -> E[p1] := cc :: E[p1]
| {t} ⊆ p =) p1 ⊆ p2 -> E[p1] := cc :: E[p1];
                        E[p] := cc :: E[p];
done
```

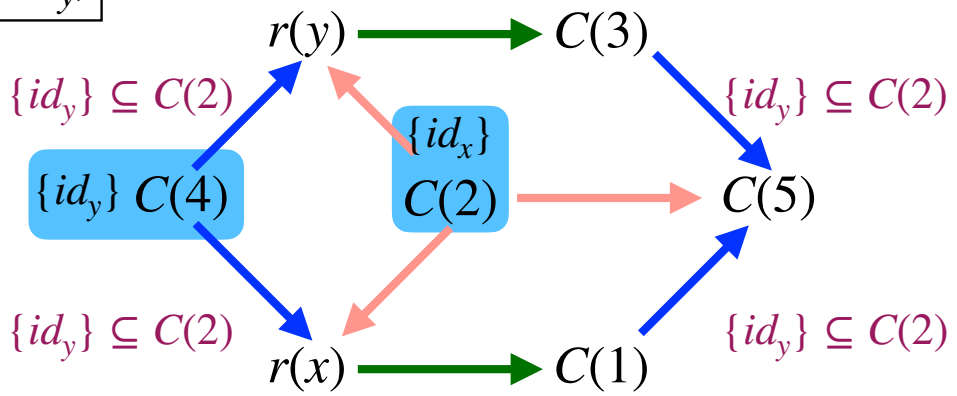
$\text{add}(C(2), id_x)$
$\text{add}(C(4), id_y)$

$D[C(2)] = \{id_x\}$
$D[C(4)] = \{id_y\}$
$D[p] = \varnothing$ for other nodes $p$

**Initial worklist**

$W = [C(4), C(2)]$

These are the nodes, whose outgoing edges should be traversed



# Back to ex. 1: step 2

```
(* Step 2: Build the graph *)
for cc in C*[[e*]] do
match cc with
| {t} ⊆ p -> add(p,{t})
|  p1 ⊆ p2 -> E[p1] := cc :: E[p1]
| {t} ⊆ p =) p1 ⊆ p2 -> E[p1] := cc :: E[p1];
                        E[p] := cc :: E[p];
done
```

$\{id_x\} \subseteq C(2)$
$\{id_y\} \subseteq C(4)$
$r(x) \subseteq C(1)$
$r(y) \subseteq C(3)$

$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$
$\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$
$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$
$\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$

$E[r(x)] := \{r(x) \subseteq C(1)\} :: [\,]$
$E[r(y)] := \{r(y) \subseteq C(3)\} :: [\,]$

$E[p]$: list of the constraints that $p$ may affect

$E[C(4)] := \{\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)\} :: []$
$E[C(2)] := \{\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)\} :: []$
$E[C(1)] := \{\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)\} :: []$
$E[C(2)] := \{\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)\} :: E[C(2)]$
$E[C(4)] = \{\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)\}$
$E[C(2)] = \{\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)\} :: E[C(2)]$
$E[C(3)] = \{\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)\}$
$E[C(2)] = \{\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)\} :: E[C(2)]$

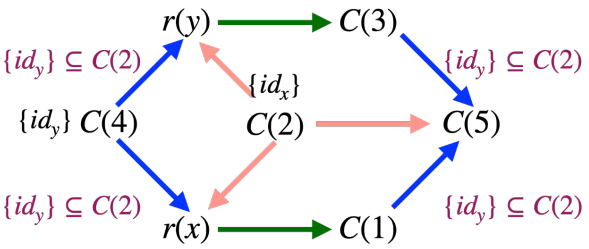
# Back to ex. 1: after Step 2

$W = [C(4), C(2)]$

list of the constraints that p may affect

Current abstract value

| $p$ | D[$p$] | E[$p$] |
|---|---|---|
| C(1) | $\emptyset$ | [$\mathsf{id}_x \subseteq \mathsf{C}(2) \Rightarrow \mathsf{C}(1) \subseteq \mathsf{C}(5)$] |
| C(2) | $\mathsf{id}_x$ | [$\mathsf{id}_y \subseteq \mathsf{C}(2) \Rightarrow \mathsf{C}(3) \subseteq \mathsf{C}(5)$, $\mathsf{id}_y \subseteq \mathsf{C}(2) \Rightarrow \mathsf{C}(4) \subseteq \mathsf{r}(\mathtt{y})$, $\mathsf{id}_x \subseteq \mathsf{C}(2) \Rightarrow \mathsf{C}(1) \subseteq \mathsf{C}(5)$, $\mathsf{id}_x \subseteq \mathsf{C}(2) \Rightarrow \mathsf{C}(4) \subseteq \mathsf{r}(\mathtt{x})$] |
| C(3) | $\emptyset$ | [$\mathsf{id}_y \subseteq \mathsf{C}(2) \Rightarrow \mathsf{C}(3) \subseteq \mathsf{C}(5)$] |
| C(4) | $\mathsf{id}_y$ | [$\mathsf{id}_y \subseteq \mathsf{C}(2) \Rightarrow \mathsf{C}(4) \subseteq \mathsf{r}(\mathtt{y})$, $\mathsf{id}_x \subseteq \mathsf{C}(2) \Rightarrow \mathsf{C}(4) \subseteq \mathsf{r}(\mathtt{x})$] |
| C(5) | $\emptyset$ | [ ] |
| r(x) | $\emptyset$ | [$\mathsf{r}(\mathtt{x}) \subseteq \mathsf{C}(1)$] |
| r(y) | $\emptyset$ | [$\mathsf{r}(\mathtt{y}) \subseteq \mathsf{C}(3)$] |



# The algorithm

We have now built the graph and initialized the analysis
Next, we propagate abstract values along edges, through Step 3, until we reach a fixpoint

This is a standard worklist-based fixpoint computation

# The algorithm pseudocode (3,4)

```
(* Step 3: Iteration *)
while W ≠ nil do
q:= head(W); W := tail(W);
for cc in E[q] do
case cc of
 p1 ⊆ p2: add(p2,D[p1]);// propagate
         // all values from p1 to p2
{t} ⊆ p ⇒ p1 ⊆ p2:
    if t ∈ D(p) then add(p2,D[p1]);
    // // activate conditional constraint
```

Iterative fixpoint

It propagates D[p1] in D[p2]

if t ∈ D[p] then propagate D[p1] in D[p2]

```
(* Step 4: Recording the solution *)
for ℓ ∈ Lab* do Ĉ(ℓ): = D[C(ℓ)];
for x ∈ Var*, do ρ̂(x):= D[r(x)];
```

# The algorithm: step 3

Continue propagating the information from one data field to another, along edges, as long as $W$ is non-empty

$E[q]$: list of the constraints that $q$ may affect

We extract the head element $q$ of $W$ and for all constraints $cc$ in $E[q]$

- $cc = p_1 \subseteq p_2$: If we have an edge $(p_1, p_2)$ in our graph , then the value set of $p_1$ must flow into the value set of $p_2$ (add$(p_2, D[p_1])$)
- $cc = \{t\} \subseteq p \Rightarrow p_1 \subseteq p_2$: If the trigger is true ($t \in D[p]$), then also in the corresponding edge the value set flows (add$(p_2, D[p_1])$)

# Back to ex. 1: 1st iteration (C(4))

$W = [C(4), C(2)]$

Take $C(4)$ from the top

$D[C(4)] = \{id_y\}$

$$\{id_x\} \subseteq C(2)$$
$$\{id_y\} \subseteq C(4)$$
$$r(x) \subseteq C(1)$$
$$r(y) \subseteq C(3)$$

$$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$$
$$\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$$
$$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$$
$$\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$$

$E(C(4)) =$

$\{\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x),$

$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)\}$

Constraint is triggered since

$id_x \in D[C(2)] = \{id_x\}$:

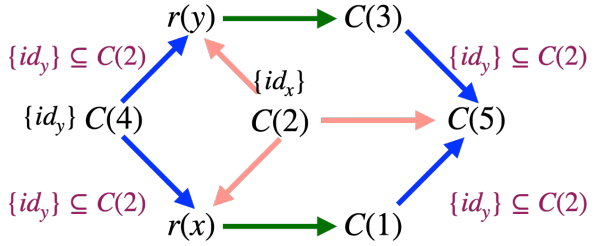
$\texttt{add}(r(x), D[C(4)])$

propagate $D[C(4)]$ to $r(x)$

- incorporate $\{id_y\}$ into $D[r(x)]$
- add $r(x)$ to $W$

$D[r(x)] := \{id_y\}$

$W := [r(x), C(2)]$




```
(* Step 3: Iteration *)
while W ≠ nil do
q:= head(W); W := tail(W);
for cc in E[q] do
case cc of
 p1 ⊆ p2: add(p2,D[p1]);
{t} ⊆ p ⇒ p1 ⊆ p2:
    if t ∈ D[p] then add(p2,D[p1]);
```

# Back to ex. 1: 1st iteration (C(4))

$W = [C(4), C(2)]$

Take $C(4)$ from the top

$D[C(4)] = \{id_y\}$

$\{id_x\} \subseteq C(2)$
$\{id_y\} \subseteq C(4)$
$r(x) \subseteq C(1)$
$r(y) \subseteq C(3)$

$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$
$\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$
$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$
$\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$

$E(C(4)) =$
$\{\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x),$
$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)\}$

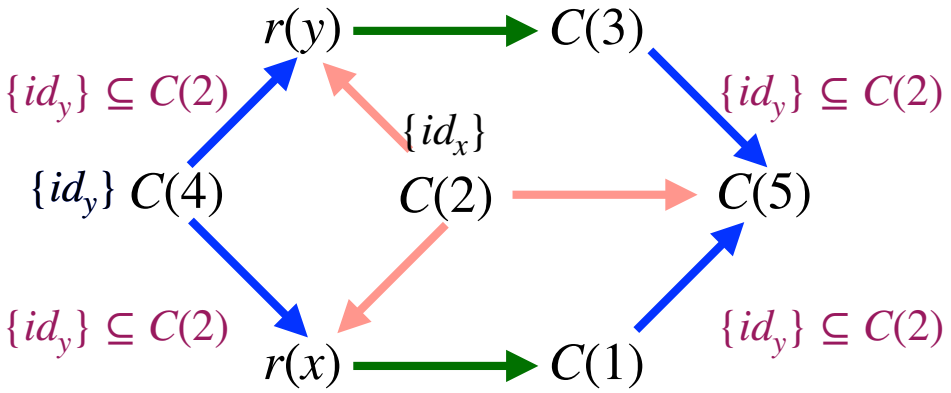
Constraint is triggered since
$id_x \in D[C(2)] = \{id_x\}$:
$\mathtt{add}(r(x), D[C(4)])$

propagate $D[C(4)]$ to $r(x)$

- incorporate $\{id_y\}$ into $D[r(x)]$
- add $r(x)$ to $W$

$D[r(x)] := \{id_y\}$

$W := [r(x), C(2)]$



# Back to ex. 1: 1st iteration (C(4))

$W = [C(4), C(2)]$

Take $C(4)$ from the top

$D[C(4)] = \{id_y\}$

$$\{id_x\} \subseteq C(2)$$
$$\{id_y\} \subseteq C(4)$$
$$r(x) \subseteq C(1)$$
$$r(y) \subseteq C(3)$$

$$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$$
$$\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$$
$$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$$
$$\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$$

$E(C(4)) =$

$\{\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x),$

$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)\}$

Constraint is not triggered since

$id_Y \notin D[C(2)] = \{id_x\}$

hence no propagation from

$D[C(4)]$ to $r(y)$

```
(* Step 3: Iteration *)
while W ≠ nil do
q:= head(W); W := tail(W);
for cc in E[q] do
case cc of
 p1 ⊆ p2: add(p2,D[p1]);
{t} ⊆ p ⇒ p1 ⊆ p2:
    if t ∈ D[p] then add(p2,D[p1]);
```

# Back to ex. 1: 2nd iteration (r(x))

$W = [r(x), C(2)]$

Take $r(x)$ from the top

$D[r(x)] = \{id_y\}$

$$\{id_x\} \subseteq C(2)$$
$$\{id_y\} \subseteq C(4)$$
$$r(x) \subseteq C(1)$$
$$r(y) \subseteq C(3)$$

$$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$$
$$\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$$
$$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$$
$$\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$$

$E(r(x)) =$

$\{\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)\}$

Constraint is triggered since

$id_x \in D[C(2)] = \{id_x\}$:

$\text{add}(C(1), D[r(x)])$

propagate $D[r(x)]$ to $C(1)$

$D[C(1)] = \{id_y\}$

$W = [C(1), C(2)]$

```
(* Step 3: Iteration *)
while W ≠ nil do
q:= head(W); W := tail(W);
for cc in E[q] do
case cc of
 p1 ⊆ p2: add(p2,D[p1]);
{t} ⊆ p ⇒ p1 ⊆ p2:
    if t ∈ D(p) then add(p2,D[p1]);
```

# Back to ex. 1: 2nd iteration (r(x))

$W = [r(x), C(2)]$

Take $r(x)$ from the top

$D[r(x)] = \{id_y\}$

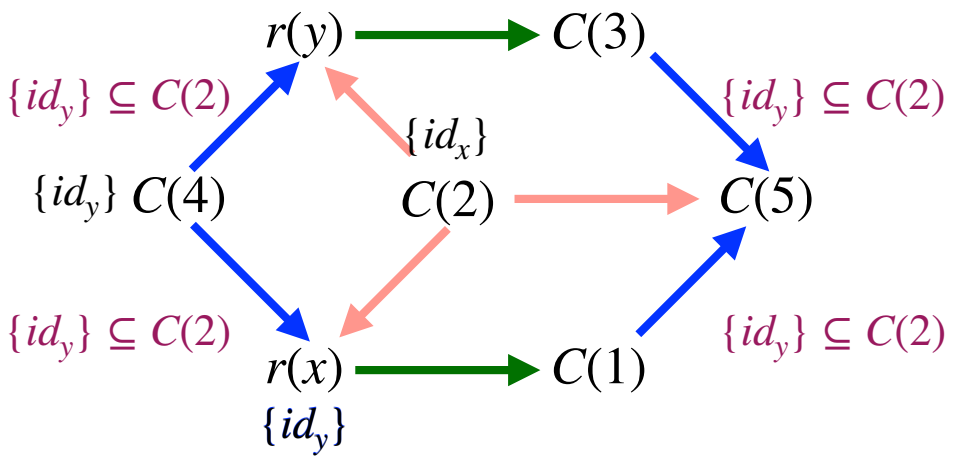
$\{id_x\} \subseteq C(2)$
$\{id_y\} \subseteq C(4)$
$r(x) \subseteq C(1)$
$r(y) \subseteq C(3)$

$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$
$\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$
$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$
$\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$

$E(r(x)) =$
$\{\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)\}$

Constraint is triggered since
$id_x \in D[C(2)] = \{id_x\}$:
add$(C(1), D[r(x)])$

propagate $D[r(x)]$ to $C(1)$




$D[C(1)] = \{id_y\}$

$W = [C(1), C(2)]$

# Back to ex. 1: 3rd iteration (C(1))

$W = [C(1), C(2)]$

Take $C(1)$ from the top

$D[C(1)] = \{id_y\}$

$$\{id_x\} \subseteq C(2)$$
$$\{id_y\} \subseteq C(4)$$
$$r(x) \subseteq C(1)$$
$$r(y) \subseteq C(3)$$

$$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$$
$$\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$$
$$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$$
$$\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$$

$E(C(1)) =$
$\{\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)\}$

Constraint is triggered since
$id_x \in D[C(2)]$:
$\text{add}(C(5), D[C(1)])$

propagate $D[C(1)]$ to $C(5)$

$D[C(5)] = \{id_x\}$
$W = [C(5), C(2)]$

```
(* Step 3: Iteration *)
while W ≠ nil do
q:= head(W); W := tail(W);
for cc in E[q] do
case cc of
 p1 ⊆ p2: add(p2,D[p1]);
{t} ⊆ p ⇒ p1 ⊆ p2:
    if t ∈ D(p] then add(p2,D[p1]);
```

# Back to ex. 1: 3rd iteration (C(1))

$W = [C(1), C(2)]$

Take $C(1)$ from the top

$D[C(1)] = \{id_y\}$

$\{id_x\} \subseteq C(2)$
$\{id_y\} \subseteq C(4)$
$r(x) \subseteq C(1)$
$r(y) \subseteq C(3)$

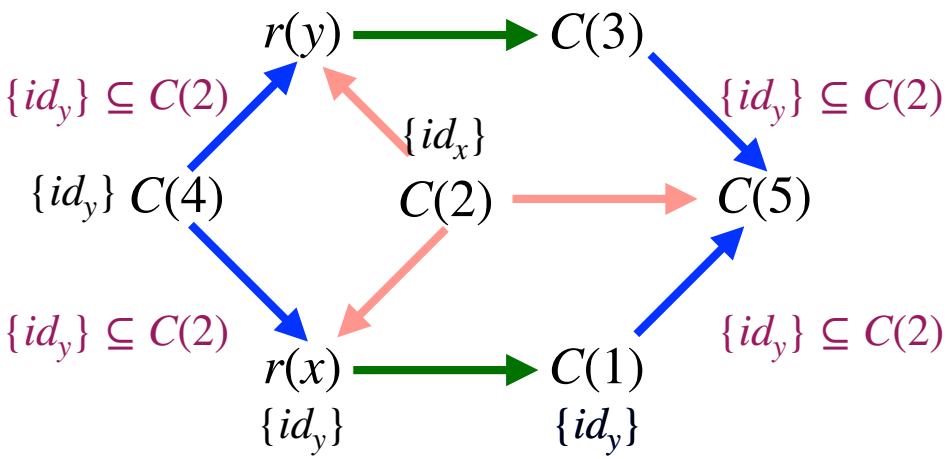
$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$
$\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$
$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$
$\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$

$E(C(1)) =$
$\{\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)\}$

Constraint is triggered since
$id_x \in D[C(2)]$:
$\text{add}(C(5), D[C(1)])$

propagate $D[C(1)]$ to $C(5)$

$D[C(5)] = \{id_x\}$
$W = [C(5), C(2)]$



# Back to ex. 1: 4th iteration (C(5))

$W = [C(5), C(2)]$

Take $C(5)$ from the top

$D[C(1)] = \{id_x\}$

$$\{id_x\} \subseteq C(2)$$
$$\{id_y\} \subseteq C(4)$$
$$r(x) \subseteq C(1)$$
$$r(y) \subseteq C(3)$$

$$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$$
$$\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$$
$$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$$
$$\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$$

$E(C(5)) = \emptyset$ → No update → $W = [C(2)]$

# Back to ex. 1: 4th iteration (C(5))

$W = [C(5), C(2)]$

Take $C(5)$ from the top

$D[C(1)] = \{id_x\}$

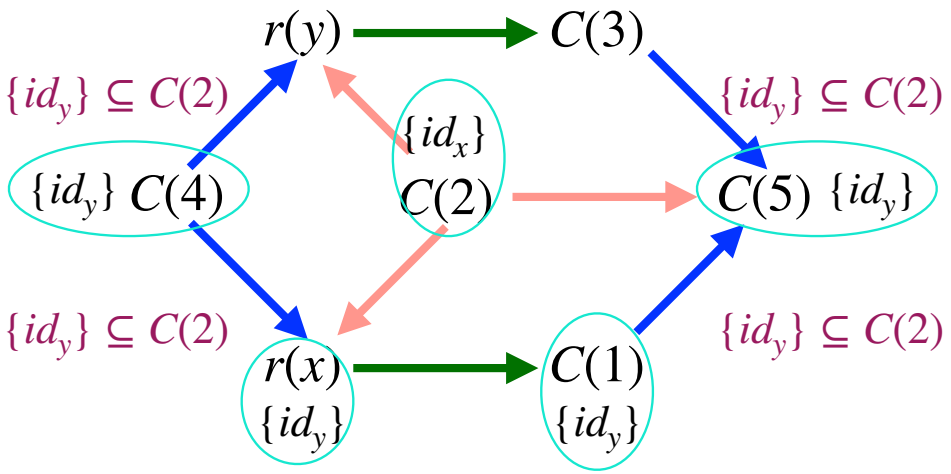
$\{id_x\} \subseteq C(2)$
$\{id_y\} \subseteq C(4)$
$r(x) \subseteq C(1)$
$r(y) \subseteq C(3)$

$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$
$\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$
$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$
$\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$

$E(C(5)) = \emptyset$

No update

$W = [C(2)]$




| W | [C(2)] |
|---|---|
| q | D[q] |
| C(1) | $id_y$ |
| C(2) | $id_x$ |
| C(3) | ∅ |
| C(4) | $id_y$ |
| C(5) | $id_y$ |
| r(x) | $id_y$ |
| r(y) | ∅ |

# Back to ex. 1: 5th iteration (C(2))

$W = [C(2)]$

Take $C(2)$ from the top

$D[C(2)] = \{id_x\}$

$E(C(2)) = \{$

$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$

$\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$

$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$

$\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$

$\}$

$\{id_x\} \subseteq C(2)$
$\{id_y\} \subseteq C(4)$
$r(x) \subseteq C(1)$
$r(y) \subseteq C(3)$

$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$
$\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$
$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$
$\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$

**Only** constraints for $id_x$ are triggered

$\texttt{add}(r(x), D[C(4)])$ ✓

$\texttt{add}(C(5), D[C(1)])$ ✓

No new information is added

⇒ no node is reinserted in $W$

⇒ $W$ becomes empty

(fixpoint reached)

$W = []$

| [C(2)] | [] |
|---|---|
| D[q] | D[q] |
| $id_y$ | $id_y$ |
| $id_x$ | $id_x$ |
| ∅ | ∅ |
| $id_y$ | $id_y$ |
| $id_y$ | $id_y$ |
| $id_y$ | $id_y$ |
| ∅ | ∅ |

# Back to ex. 1: iteration steps

| W | [C(4), C(2)] | [r(x), C(2)] | [C(1), C(2)] | [C(5), C(2)] | [C(2)] | [] |
|---|---|---|---|---|---|---|
| q | D[q] | D[q] | D[q] | D[q] | D[q] | D[q] |
| C(1) | $\emptyset$ | $\emptyset$ | $id_y$ | $id_y$ | $id_y$ | $id_y$ |
| C(2) | $id_x$ | $id_x$ | $id_x$ | $id_x$ | $id_x$ | $id_x$ |
| C(3) | $\emptyset$ | $\emptyset$ | $\emptyset$ | $\emptyset$ | $\emptyset$ | $\emptyset$ |
| C(4) | $id_y$ | $id_y$ | $id_y$ | $id_y$ | $id_y$ | $id_y$ |
| C(5) | $\emptyset$ | $\emptyset$ | $\emptyset$ | $id_y$ | $id_y$ | $id_y$ |
| r(x) | $\emptyset$ | $id_y$ | $id_y$ | $id_y$ | $id_y$ | $id_y$ |
| r(y) | $\emptyset$ | $\emptyset$ | $\emptyset$ | $\emptyset$ | $\emptyset$ | $\emptyset$ |

fix point

# Back to ex. 1: the solution

$$((\texttt{fn x => x}^{1})^{2}\ (\texttt{fn y => y}^{3})^{4})^{5}$$

| W | [] |
|---|---|
| q | D[q] |
| C(1) | $id_y$ |
| C(2) | $id_x$ |
| C(3) | $\emptyset$ |
| C(4) | $id_y$ |
| C(5) | $id_y$ |
| r(x) | $id_y$ |
| r(y) | $\emptyset$ |

The computed solution is $(\hat{C}, \hat{\rho})$:

- $\forall \ell \in \mathbf{Lab}_* : \hat{C}(\ell) := D[C(\ell)]$
- $\forall x \in \mathbf{Var}_* : \hat{\rho}(x) := D[r(x)]$

$$\hat{C}(1) = \{id_y\}$$
$$\hat{C}(2) = \{id_x\}$$
$$\hat{C}(3) = \emptyset$$
$$\hat{C}(4) = \{id_y\}$$
$$\hat{C}(5) = \{id_y\}$$
$$\hat{\rho}(x) = \{id_y\}$$
$$\hat{\rho}(y) = \emptyset$$

# Complexity

The proof showing that the algorithm terminates can be refined to show that it takes at most $O(n^3)$ steps if the original expression $e_\star$ has size $n$. To see this recall that $\mathcal{C}_\star[\![e_\star]\!]$ contains at most $O(n)$ constraints of the form $\{t\} \subseteq p$ or $p_1 \subseteq p_2$, and at most $O(n^2)$ constraints of the form $\{t\} \subseteq p \Rightarrow p_1 \subseteq p_2$. We therefore know that the graph has at most $O(n)$ nodes and $O(n^2)$ edges and that each data field can be enlarged at most $O(n)$ times. Assuming that the operations upon $\mathsf{D}[p]$ take unit time we can perform the following calculations: step 1 takes time $O(n)$, step 2 takes time $O(n^2)$, and step 4 takes time $O(n)$; step 3 traverses each of the $O(n^2)$ edges at most $O(n)$ times and hence takes time $O(n^3)$; it follows that the overall algorithm takes no more than $O(n^3)$ basic steps.

# Correctness of the algorithm

**Proposition**

Given the input $C_*[\![e_*]\!]$, the algorithm terminates and the result $(\hat{C}, \hat{\rho})$ satisfies

$$(\hat{C}, \hat{\rho}) = \sqcap \{(\hat{C}', \hat{\rho}') \mid (\hat{C}', \hat{\rho}') \vDash_c C_*[\![e_*]\!]\}$$

and hence it is the least solution to $C_*[\![e_*]\!]$

is the least solution of the constraints

As a consequence $(\hat{C}, \hat{\rho}) \vDash_c C_*[\![e_*]\!]$, i.e., the computed solution satisfies all constraints

**Corollary**

Assume that $(\hat{C}, \hat{\rho})$ is the solution to the constraints $C_*[\![e_*]\!]$ computed by the algorithm, then $(\hat{C}, \hat{\rho}) \vDash e_*$

$(\hat{C}, \hat{\rho})$ is a valid CFA of $e_*$

Solving constraints yields a sound analysis

# Correctness of the algorithm

**Proposition**

Given the input $C_*[\![e_*]\!]$, the algorithm terminates and the result $(\hat{C}, \hat{\rho})$ satisfies

$$(\hat{C}, \hat{\rho}) = \sqcap \{(\hat{C}', \hat{\rho}') \mid (\hat{C}', \hat{\rho}') \vDash_c C_*[\![e_*]\!]\}$$

and hence it is the least solution to $C_*[\![e_*]\!]$

**Proof sketch**

**Termination**:

(The steps 1,2,4 terminate. We have just to look at step 3).

The algorithm uses a finite set of terms ($\mathbf{Term}_*$) and monotone updates.

**Soundness**:

If $(\hat{C}', \hat{\rho}') \vDash_c C_*[\![e_*]\!]\}$: the following invariant is maintained:

$D[C(\ell)] \subseteq \hat{C}'(\ell)$ and $D[r(x)] \subseteq \hat{\rho}'(x)$.

Hence $(\hat{C}, \hat{\rho}) \subseteq (\hat{C}', \hat{\rho}')$.

We must prove that $(\hat{C}, \hat{\rho}) \vDash_c C_*[\![e_*]\!]$.

# Correctness of the algorithm

**Proposition**

Given the input $C_*[\![e_*]\!]$, the algorithm terminates and the result $(\hat{C}, \hat{\rho})$ satisfies

$$(\hat{C}, \hat{\rho}) = \sqcap \{(\hat{C}', \hat{\rho}') \mid (\hat{C}', \hat{\rho}') \vDash_c C_*[\![e_*]\!]\}$$

and hence it is the least solution to $C_*[\![e_*]\!]$

**Proof sketch (cont.)**

**Soundness (cont.)**:

Assume by contradiction that some constraint $cc \in C_*[\![e_*]\!]$ is not satisfied by $(\hat{C}, \hat{\rho})$. According to the algorithm this could not be true.

$\Rightarrow$ The computed solution satisfies all $cc \in C_*[\![e_*]\!]$, i.e., $(\hat{C}, \hat{\rho}) \vDash_c cc$.

**Minimality**:

The result is included in every solution $\Rightarrow$ it is the least solution, computed as

$$\sqcap \{(\hat{C}', \hat{\rho}') \mid (\hat{C}', \hat{\rho}') \vDash_c C_*[\![e_*]\!]\}$$

# Computation vs check of acceptability

**Goal**: Compute a solution vs check that a solution is acceptable

There are applications where the ability to check $(\hat{C}, \hat{\rho}) \vDash e_*$ is more important than computing the solution

**Scenario**

*Open system* $e_0$ that uses resources of an unspecified library $e_*$

Analyses and optimizations of $e_0$ therefore rely on an abstraction $(\hat{C}, \hat{\rho})$ expressing the relevant properties about the library $e_*$, meaning that $(\hat{C}, \hat{\rho}) \vDash e_*$

**Idea**:

Use $(\hat{C}, \hat{\rho})$ as a **specification** of the library $e_*$

**Guarantee**

Any other library $e'_*$ such that $(\hat{C}, \hat{\rho}) \vDash e'_*$ can safely replace $e_*$, since the specification $(\hat{C}, \hat{\rho})$ still guarantees all required properties about the library $e'_*$

# Adding Data Flow Analysis

# CFA + DFA

We extend Control Flow Analysis with data-flow information

**Abstract values** = functions + data-flow properties

$$\hat{v} \in \widehat{\mathbf{Val}}_d = \wp(\mathbf{Term} \cup \mathbf{Data})$$

**Abstract values**

Now contain

- possible functions (as before) and
- abstract data-flow properties,(i.e., properties of booleans and integers, such as sign, constantness, or ranges of values)

**The data-flow component can be:**

- a powerset (e.g., sets of possible values)
- a complete lattice (e.g., intervals, signs, constants)

# Abstract domains

$$\hat{v} \in \widehat{\mathbf{Val}_d} = \wp(\mathbf{Term} \cup \mathbf{Data})$$

**Constants**

Each constant $c \in \mathbf{Const}$ is mapped to an element $d_c \in \mathbf{Data}$ specifying the abstract property of $c$:

$d_c$ can be thought as $\alpha(c)$ for a suitable abstraction $\alpha$

**Operators** are lifted from concrete data to abstract values, i.e., they operate on abstract properties instead of concrete values

To each operator $op \in \mathbf{Op}$, corresponds a total function

$\hat{op} : \widehat{\mathbf{Val}_d} \times \widehat{\mathbf{Val}_d} \to \widehat{\mathbf{Val}_d}$ telling how $op$ operates on abstract properties

Abstract operators over-approximate concrete operations

# Abstract operators

$$\hat{op} : \widehat{\mathbf{Val}_d} \times \widehat{\mathbf{Val}_d} \to \widehat{\mathbf{Val}_d}$$

We apply the operator to all combinations of abstract data values

Typically, $\hat{op}$ has a definition of the form

$$\hat{v}_1 \ \hat{op} \ \hat{v}_1 = \bigcup \{ d_{op}(d_1, d_2) \mid d_1 \in \hat{v}_1 \cap \mathbf{Data}, d_2 \in \hat{v}_2 \cap \mathbf{Data} \}$$

for some function $op : \mathbf{Data} \times \mathbf{Data} \to \wp(\mathbf{Data})$ specifying how the operator computes with the abstract properties of integers and booleans

This over-approximates all possible concrete computations

# Example: signs analysis

$$\mathbf{Data}_{\text{sign}} = \{\text{tt}, \text{ff}, -, 0, +\}$$

Each constant is mapped to a sign: $d_{true} = \text{tt}$ and $d_7 = +$,...

Take, e.g., $\hat{+}$ as

$$\hat{v}_1 \,\hat{+}\, \hat{v}_2 = \bigcup_{d_1 \in \hat{v}_1, d_2 \in \hat{v}_2} \{d_+(d_1, d_2)\}$$

This defines the abstract semantics of +

Abstract addition on signs

| $d_+$ | tt | ff | – | 0 | + |
|---|---|---|---|---|---|
| tt | ∅ | ∅ | ∅ | ∅ | ∅ |
| ff | ∅ | ∅ | ∅ | ∅ | ∅ |
| – | ∅ | ∅ | {–} | {–} | {–, 0, +} |
| 0 | ∅ | ∅ | {–} | {0} | {+} |
| + | ∅ | ∅ | {–, 0, +} | {+} | {+} |

For instance, if

$\hat{v}_1 = \{-, 0\}$ and $\hat{v}_2 = \{+\}$

then

$\hat{v}_1 \,\hat{+}\, \hat{v}_2 =$

$d_+(-, +) \cup d_+(0, +) = \{-, 0, +\}$

# Acceptability relation

The acceptability relation of the combined analysis has the form

$$(\hat{C}, \hat{\rho}) \vDash_d e$$

Each rule over-approximates both control-flow and data-flow

The rules are as before, but operate on enriched abstract values

**Key clauses**:

- [con]: maps constants to abstract data values
- [if]: uses abstract boolean information to select possible branches
- [op]: uses abstract operators on data

Data-flow information is propagated through the evaluation of expressions

# Clause for constants

$$[con]\ (\hat{C}, \hat{\rho}) \vDash_d c^\ell \text{ iff } \{d_c\} \subseteq \hat{C}(\ell)$$

We now track data values: the CFA records that $d_c$ is a possible value of $c$

Constants introduce abstract data into the analysis

**NEW**: constants contribute data (not functions)

**Example**

$c^\ell = 7^\ell$

CFA:

$\hat{C}(\ell) = \varnothing$

CFA+DFA:

$\hat{C}(\ell) = \{ + \}$

# Clause for conditional

$$[if]\ (\hat{C},\hat{\rho}) \vDash_d (\text{if } t_0^{\ell_0} \text{ then } t_1^{\ell_1} \text{ else } t_2^{\ell_2})^{\ell}$$

$$\text{iff } (\hat{C},\hat{\rho}) \vDash_d t_0^{\ell_0} \wedge$$

$$(d_{true} \in \hat{C}_{\ell_0} \Rightarrow ((\hat{C},\hat{\rho}) \vDash_d t_1^{\ell_1} \wedge \hat{C}(\ell_1) \subseteq \hat{C}(\ell))) \wedge$$

$$(d_{false} \in \hat{C}_{\ell_0} \Rightarrow ((\hat{C},\hat{\rho}) \vDash_d t_2^{\ell_2} \wedge \hat{C}(\ell_2) \subseteq \hat{C}(\ell)))$$

Is flow sensitive: sensitive: can determine whether true or false branches can be taken

- Increased precision: we analyze only those branches consistent with the abstract value of the condition, which now is trackable
- control-flow is now filtered by data-flow

**Example**

$e \triangleq$ (if ($3^1 > 0^2)^3$ then (fn y => $y^4)^5$ else (fn z => $z^6)^7)^8$

$\delta_{\text{true}} \sqsubseteq \hat{C}(3) \Rightarrow \hat{C}(5) \subseteq \hat{C}(8)$... $\delta_{\text{false}} \not\sqsubseteq \hat{C}(3) \Rightarrow$

# Clause for binary operation

$$[op]\ (\hat{C},\hat{\rho}) \vDash_d (t_1^{\ell_1}\ \text{op}\ t_2^{\ell_2})^{\ell} \text{ iff } (\hat{C},\hat{\rho}) \vDash_d t_1^{\ell_1} \wedge (\hat{C},\hat{\rho}) \vDash_d t_2^{\ell_2} \wedge \hat{C}(\ell_1)\ \hat{op}\ \hat{C}(\ell_2) \subseteq \hat{C}(\ell)$$

**NOW**: we do track data values

This rule applies abstract operators to propagate data-flow information

Data-flow is computed locally using abstract operators

**Example**

$e \triangleq (7^1 + 4^2)^3$

CFA:

$\hat{C}(3) = \varnothing \ldots$

CFA +DFA:

$\hat{C}(1)\ \hat{+}\ \hat{C}(2) \subseteq \hat{C}(3)$ if $\hat{C}(1)$ and $\hat{C}(2)$ contain $\{+\}$ then $\hat{C}(3)$ contains $\{+\}$

# To recap

- [con]: introduces data (abstract values)
- [if]: uses abstract values to guide control-flow
- [op]: computes data-flow using abstract values

# CFA with abstract values as powersets

$$
\begin{array}{ll}
[con] & (\widehat{\mathsf{C}},\widehat{\rho}) \models_d c^{\ell} \text{ iff } \{d_c\} \subseteq \widehat{\mathsf{C}}(\ell) \\
[var] & (\widehat{\mathsf{C}},\widehat{\rho}) \models_d x^{\ell} \text{ iff } \widehat{\rho}(x) \subseteq \widehat{\mathsf{C}}(\ell) \\
[fn] & (\widehat{\mathsf{C}},\widehat{\rho}) \models_d (\texttt{fn } x \texttt{ => } e_0)^{\ell} \text{ iff } \{\texttt{fn } x \texttt{ => } e_0\} \subseteq \widehat{\mathsf{C}}(\ell) \\
[fun] & (\widehat{\mathsf{C}},\widehat{\rho}) \models_d (\texttt{fun } f\ x \texttt{ => } e_0)^{\ell} \text{ iff } \{\texttt{fun } f\ x \texttt{ => } e_0\} \subseteq \widehat{\mathsf{C}}(\ell) \\
[app] & (\widehat{\mathsf{C}},\widehat{\rho}) \models_d (t_1^{\ell_1}\ t_2^{\ell_2})^{\ell} \\
 & \quad \text{iff } (\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_1^{\ell_1} \wedge (\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_2^{\ell_2} \wedge \\
 & \quad (\forall(\texttt{fn } x \texttt{ => } t_0^{\ell_0}) \in \widehat{\mathsf{C}}(\ell_1) : \\
 & \qquad (\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_0^{\ell_0} \wedge \\
 & \qquad \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\rho}(x) \wedge \widehat{\mathsf{C}}(\ell_0) \subseteq \widehat{\mathsf{C}}(\ell)) \wedge \\
 & \quad (\forall(\texttt{fun } f\ x \texttt{ => } t_0^{\ell_0}) \in \widehat{\mathsf{C}}(\ell_1) : \\
 & \qquad (\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_0^{\ell_0} \wedge \\
 & \qquad \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\rho}(x) \wedge \widehat{\mathsf{C}}(\ell_0) \subseteq \widehat{\mathsf{C}}(\ell) \wedge \\
 & \qquad \{\texttt{fun } f\ x \texttt{ => } t_0^{\ell_0}\} \subseteq \widehat{\rho}(f)) \\
[if] & (\widehat{\mathsf{C}},\widehat{\rho}) \models_d (\texttt{if } t_0^{\ell_0} \texttt{ then } t_1^{\ell_1} \texttt{ else } t_2^{\ell_2})^{\ell} \\
 & \quad \text{iff } (\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_0^{\ell_0} \wedge \\
 & \quad (d_{\texttt{true}} \in \widehat{\mathsf{C}}(\ell_0) \Rightarrow ((\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_1^{\ell_1} \wedge \widehat{\mathsf{C}}(\ell_1) \subseteq \widehat{\mathsf{C}}(\ell))) \wedge \\
 & \quad (d_{\texttt{false}} \in \widehat{\mathsf{C}}(\ell_0) \Rightarrow ((\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_2^{\ell_2} \wedge \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\mathsf{C}}(\ell))) \\
[let] & (\widehat{\mathsf{C}},\widehat{\rho}) \models_d (\texttt{let } x = t_1^{\ell_1} \texttt{ in } t_2^{\ell_2})^{\ell} \\
 & \quad \text{iff } (\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_1^{\ell_1} \wedge (\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_2^{\ell_2} \wedge \\
 & \quad \widehat{\mathsf{C}}(\ell_1) \subseteq \widehat{\rho}(x) \wedge \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\mathsf{C}}(\ell) \\
[op] & (\widehat{\mathsf{C}},\widehat{\rho}) \models_d (t_1^{\ell_1}\ op\ t_2^{\ell_2})^{\ell} \\
 & \quad \text{iff } (\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_1^{\ell_1} \wedge (\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_2^{\ell_2} \wedge \\
 & \quad \widehat{\mathsf{C}}(\ell_1)\ \widehat{\mathsf{op}}\ \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\mathsf{C}}(\ell)
\end{array}
$$

# Example 3

$$e \triangleq (\texttt{let f = (fn x => (if (x}^{1} \texttt{ > 0}^{2})^{3} \texttt{ then (fn y => y}^{4})^{5} \texttt{ else (fn z => 25}^{6})^{7})^{8})^{9} \texttt{ in ((f}^{10} \texttt{ 3}^{11})^{12} \texttt{ 0}^{13})^{14})^{15}$$

**Pure 0-CFA**:

it cannot determine that the `else` branch is *unreachable*

**CFA + Sign Analysis**:

- since $\texttt{x} \in \{+\}$, the condition $\texttt{x > 0}$ is always true
- `else` branch is unreachable
- $\hat{C}(12) = \{ \texttt{fn y => y}^{4} \}$, i.e., only $\texttt{fn y => y}^{4}$ flows to label 12

# Example 3

$$e \triangleq (\texttt{let f = (fn x => (if (x}^{1} \texttt{ > 0}^{2})^{3} \texttt{ then (fn y => y}^{4})^{5} \texttt{ else (fn z => 25}^{6})^{7})^{8})^{9} \texttt{ in ((f}^{10} \texttt{ 3}^{11})^{12} \texttt{ 0}^{13})^{14})^{15}$$

At label 12:

- Pure CFA: $\hat{C}(12) = \{\texttt{fn y => y}^{4}, \texttt{fn z => 25}^{6}\}$
- With signs: $\hat{C}(12) = \{\texttt{fn y => y}^{4}\}$

Note, in particular, that

- x is always positive: $\hat{\rho}(\texttt{x}) = \{+\}$
- condition is always true: $\hat{C}(3) = \{\texttt{tt}\}$
- else branch is unreachable: $\hat{C}(7) = \varnothing$
- e will evaluate to a value with property {0}

| | Pure CFA $(\hat{C}, \hat{\rho})$ | CFA + Sign Analysis $(\hat{C}, \hat{\rho})$ |
|---|---|---|
| 1 | ∅ | {+} |
| 2 | ∅ | {0} |
| 3 | ∅ | {tt} |
| 4 | ∅ | {0} |
| 5 | {fn y => $y^4$} | {fn y => $y^4$} |
| 6 | ∅ | ∅ |
| 7 | {fn z => $25^6$} | ∅ |
| 8 | {fn y => $y^4$, fn z => $25^6$} | {fn y => $y^4$} |
| 9 | {fn x => $(\cdots)^8$} | {fn x => $(\cdots)^8$} |
| 10 | {fn x => $(\cdots)^8$} | {fn x => $(\cdots)^8$} |
| 11 | ∅ | {+} |
| 12 | {fn y => $y^4$, fn z => $25^6$} | {fn y => $y^4$} |
| 13 | ∅ | {0} |
| 14 | ∅ | {0} |
| 15 | ∅ | {0} |
| f | {fn x => $(\cdots)^8$} | {fn x => $(\cdots)^8$} |
| x | ∅ | {+} |
| y | ∅ | {0} |
| z | ∅ | ∅ |

# Observation

The combined analysis does not directly yield a valid 0-CFA solution

**Reason**

data-flow information restricts the control flow

**Intuition**

If the condition is known to be false (or true), some branches are not analyzed at all

Even if we restrict to terms:

- $\hat{C}'(\ell) = \hat{C}(\ell) \cap \mathbf{Term}$ and
- $\hat{\rho}'(x) = \hat{\rho}(x) \cap \mathbf{Term}$

this does not guarantee a valid CFA solution, i.e.,

$(\hat{C}, \hat{\rho}) \models_d e$ does not guarantee that $(\hat{C}', \hat{\rho}') \models e$